9/24/2026

LAB NO 03

Operating System

Instructor: Ms. Muneeba Banaras

Submitted By: Wasif Ali

Reg ID: 245187

Class: BSCS F'24 B

LAB TASK

Lab Activity 1 — Create and Execute a C Program

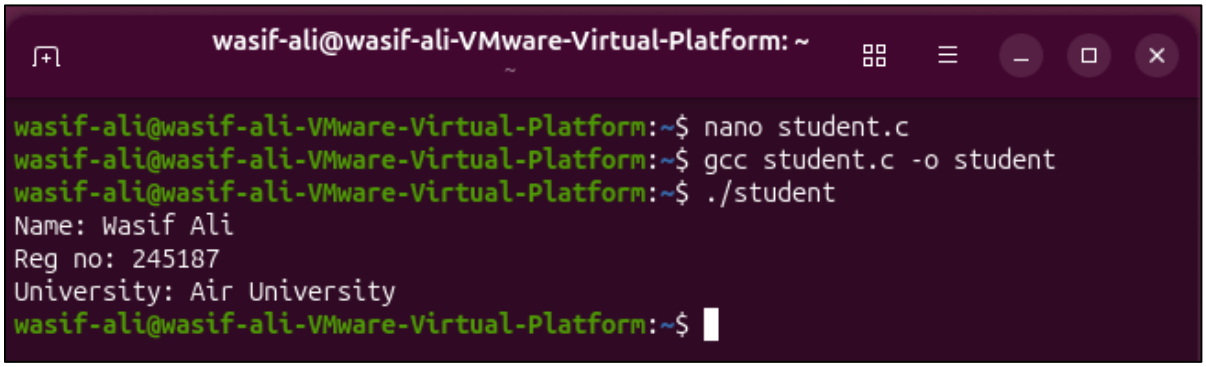
Solve a C program that displays your **name, registration number, and university**.

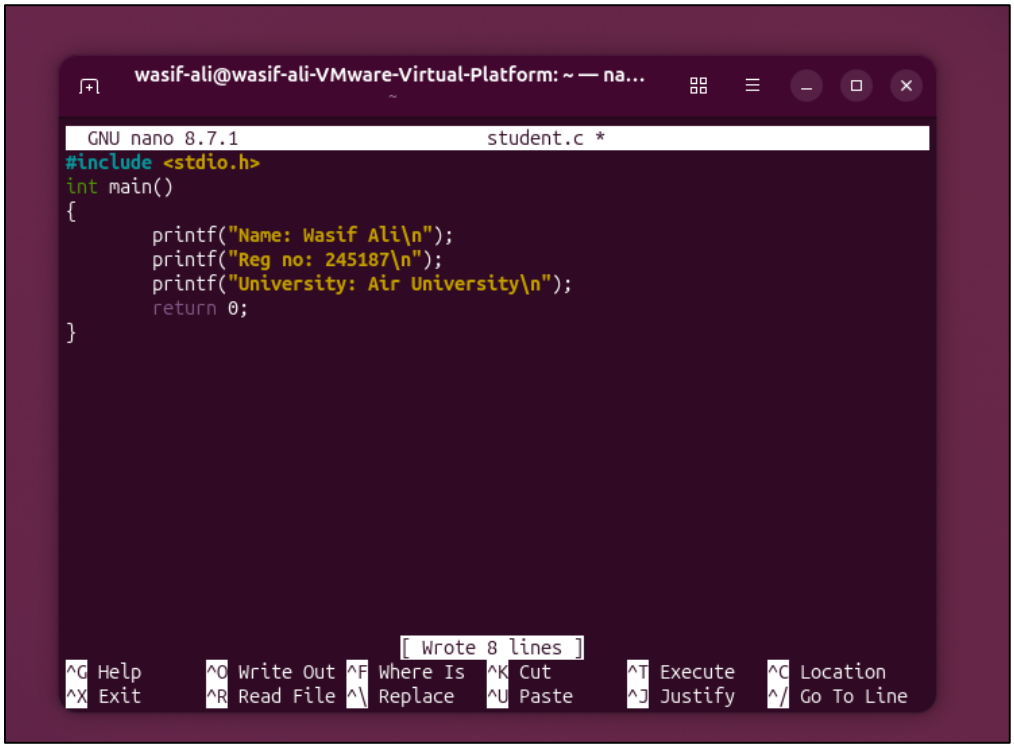
1. **Create** a C source file named student.c.
2. **Insert** the required C program.
3. **Save** the source file.
4. **Compile** the program using the gcc compiler.
5. **Modify** the program if a compilation error occurs.
6. **Execute** the compiled program.
7. **Verify** the displayed output.

Working:

In this activity, we created a C source file named student.c using the nano editor. We wrote a program using the printf function to print personal details including name, registration number, and university name. Finally, we compiled the program using the gcc compiler into an executable file named student and executed it in the terminal to verify the output.

Commands and Output:





Lab Activity 2 — Create and Execute a C++ Program

Solve a C++ program that displays your **name, registration number, and university**.

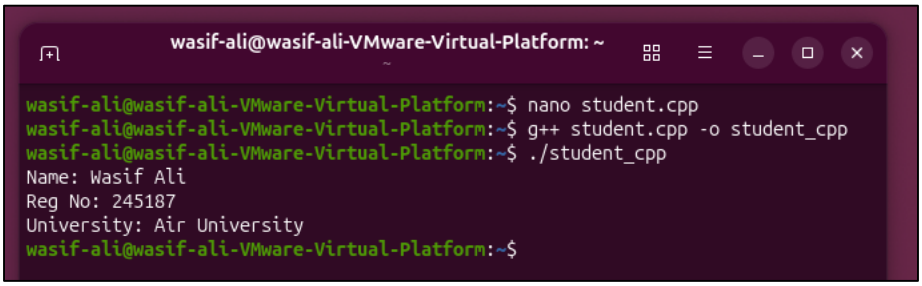
1. **Create** a C++ source file named student.cpp.
2. **Insert** the required C++ program.
3. **Apply** cout statements to display the information.
4. **Save** the program.
5. **Compile** the program using g++.
6. **Execute** the compiled program.
7. **Verify** the output.

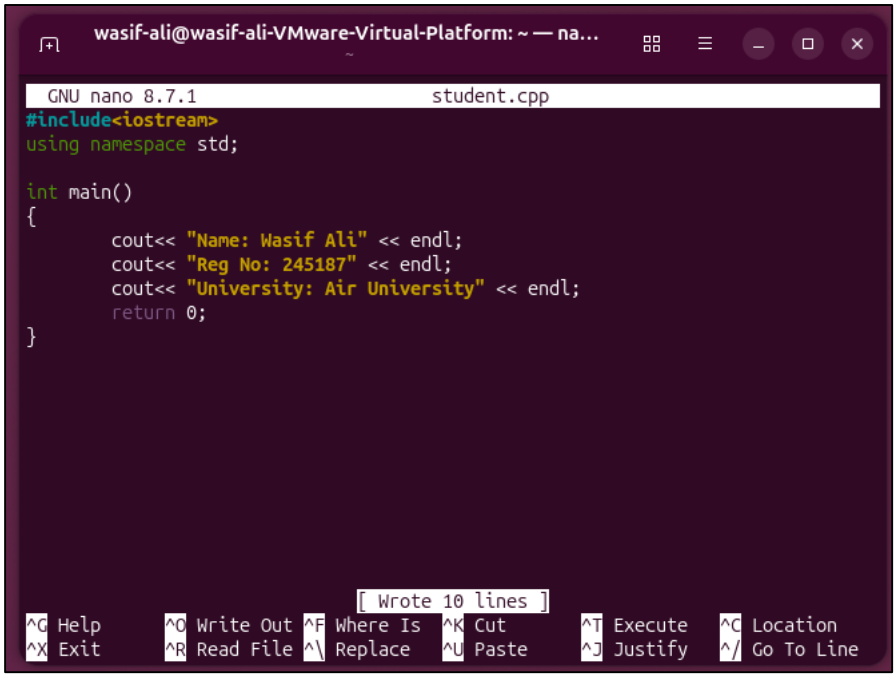
Working:

In this activity, we created a C++ source file named student.cpp. We utilized the standard cout output stream object to print personal information on the terminal.

We then compiled the C++ code using the g++ compiler to create an executable binary and ran the program to display the result.

Output and Result:





Lab Activity 3 — Arithmetic Operations in C and C++

Solve programs in **both C and C++** that take two numbers from the user and display their sum, difference, product, and quotient.

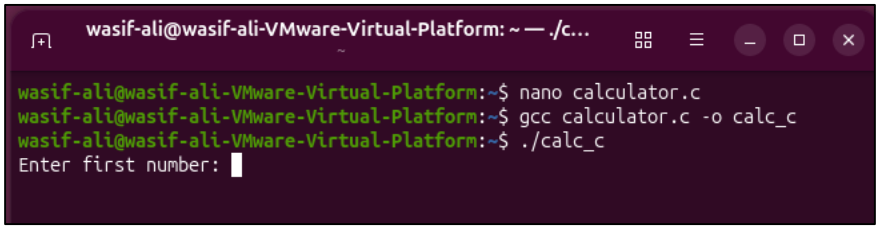
Instructions

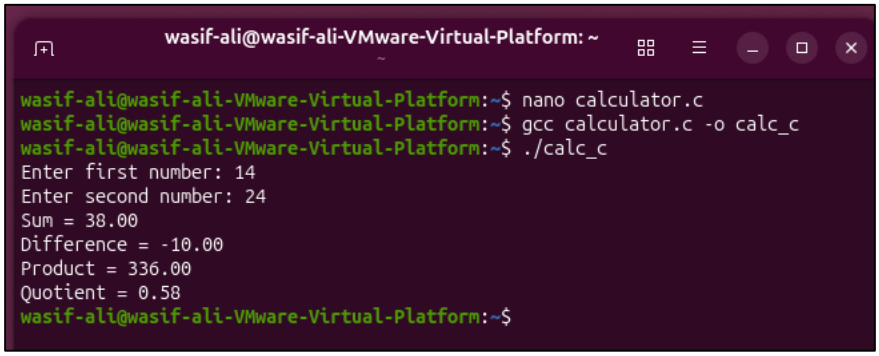
1. **Create** a C file named calculator.c.
2. **Create** a C++ file named calculator.cpp.
3. **Insert** the required code in both files.
4. **Apply** arithmetic operators for the required calculations.
5. **Compile** the C program using gcc.
6. **Compile** the C++ program using g++.
7. **Execute** both programs.
8. **Compare** their outputs.

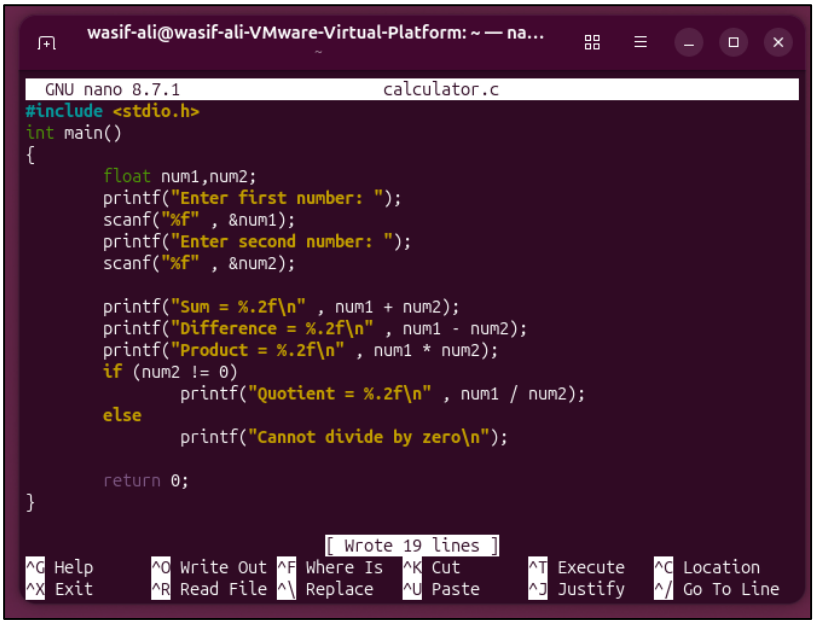
Working:

In this activity, we developed basic calculator programs in both C (calculator.c) and C++ (calculator.cpp). Both programs prompt the user to input two numeric values and calculate their sum, difference, product, and quotient. We compiled the C file using gcc and the C++ file using g++. Executing both programs confirmed that both languages yield identical mathematical output.

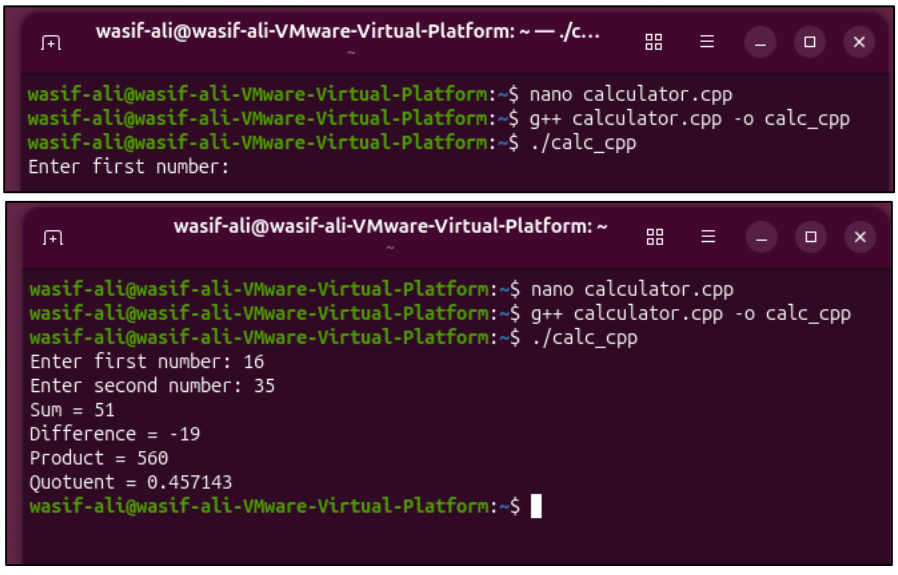
Commands & Code (C Program):

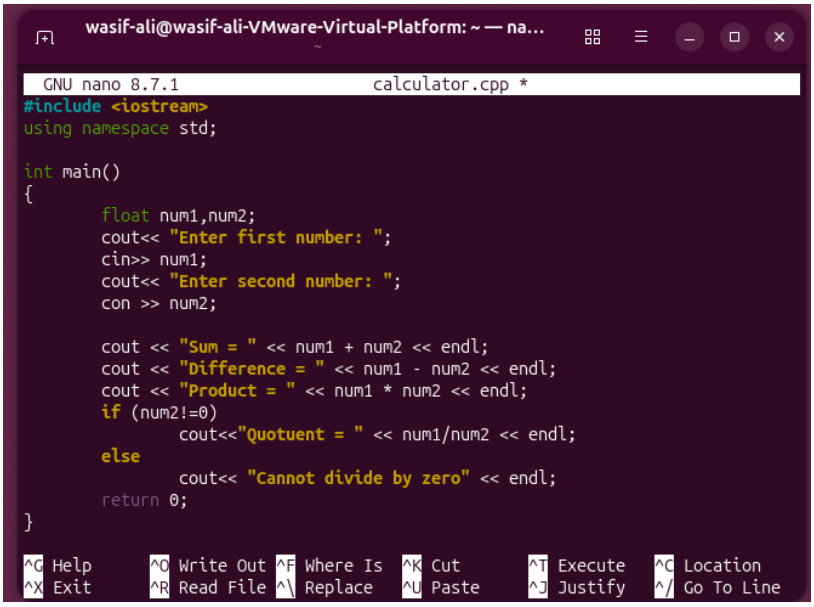






Commands & Code (C++ Program):





Lab Activity 4 — Conditional Statements

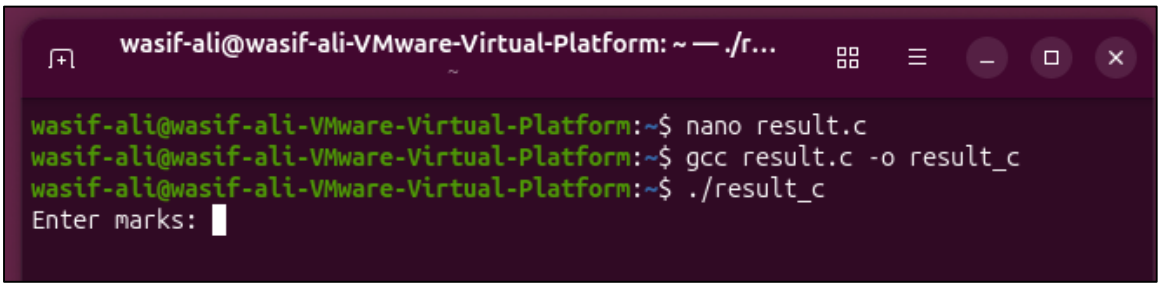
Create **C and C++ programs** to determine whether a student has passed or failed.

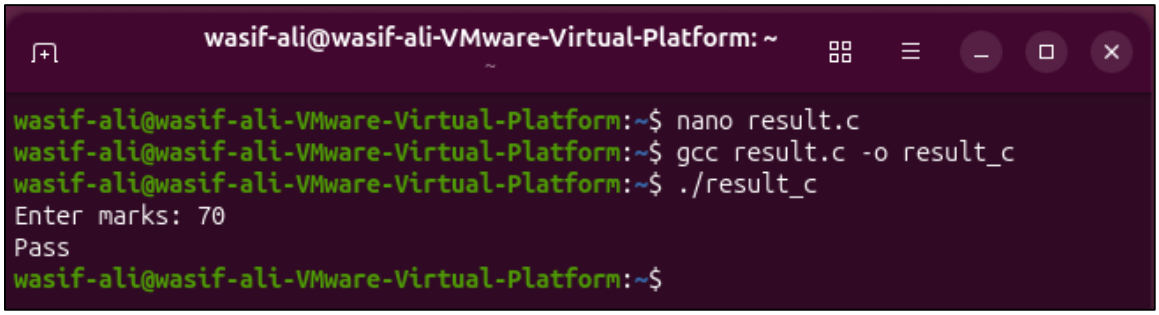
1. **Create** result.c.
2. **Create** result.cpp.
3. **Insert** an appropriate program in both files.
4. **Apply** an if-else statement.
5. **Set** the passing marks to 50.
6. **Compile** both programs using their respective compilers.
7. **Execute** both programs.
8. **Verify** the results for different marks.

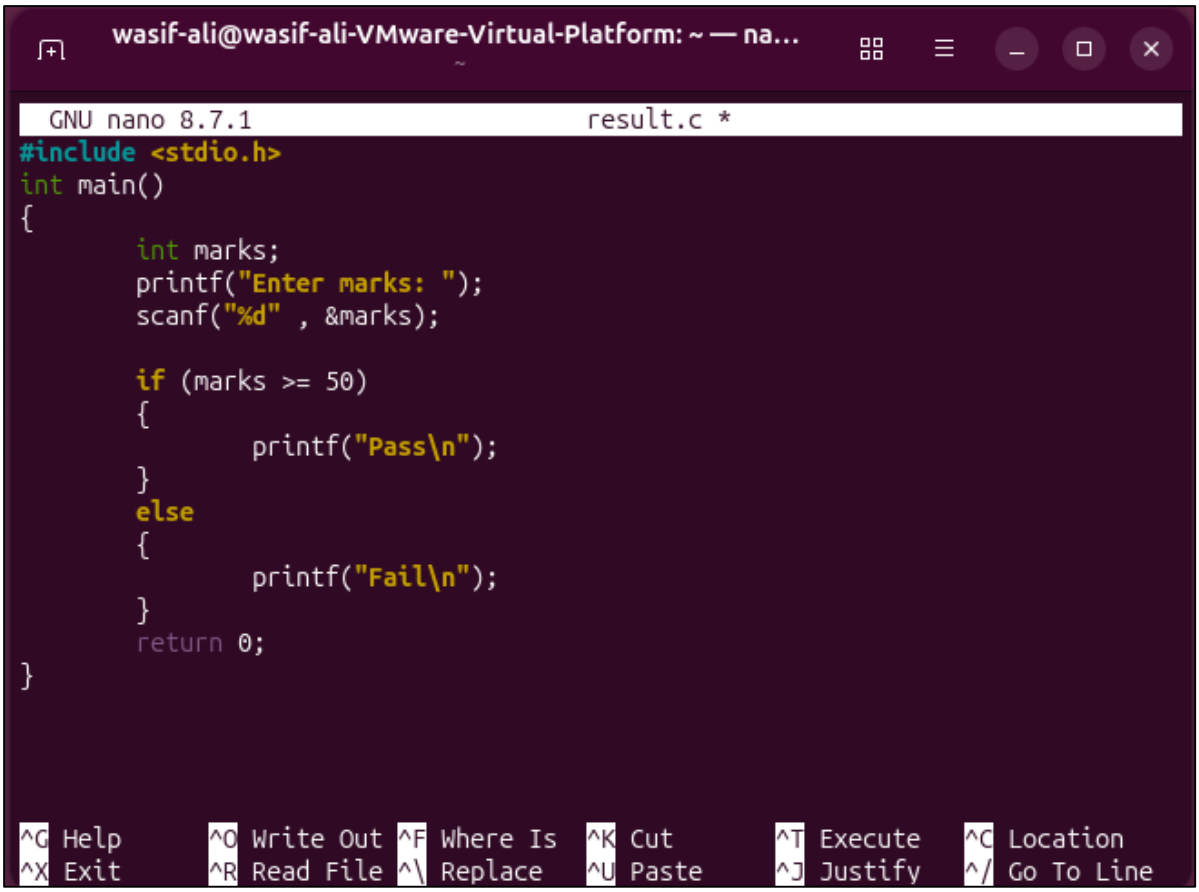
Working:

In this activity, we implemented conditional control structures in C (result.c) and C++ (result.cpp). The programs ask the user for student marks and evaluate them using an if-else condition with a passing mark threshold set to 50. Inputs of 50 or above display a "Pass" message, while inputs below 50 display "Fail".

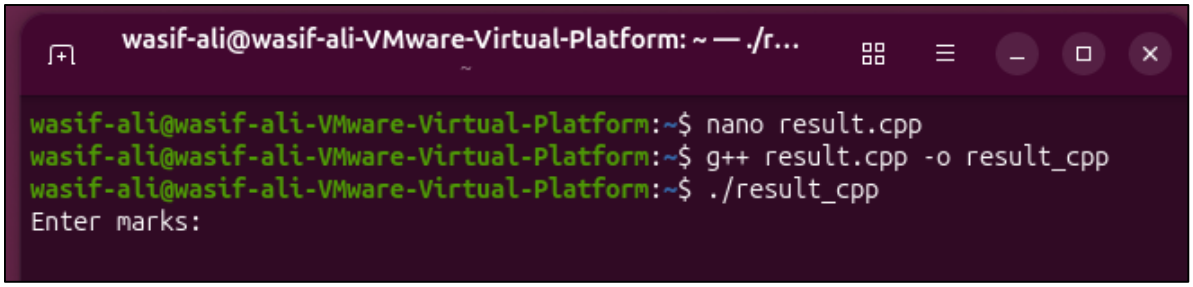
Commands & Code (C Program):

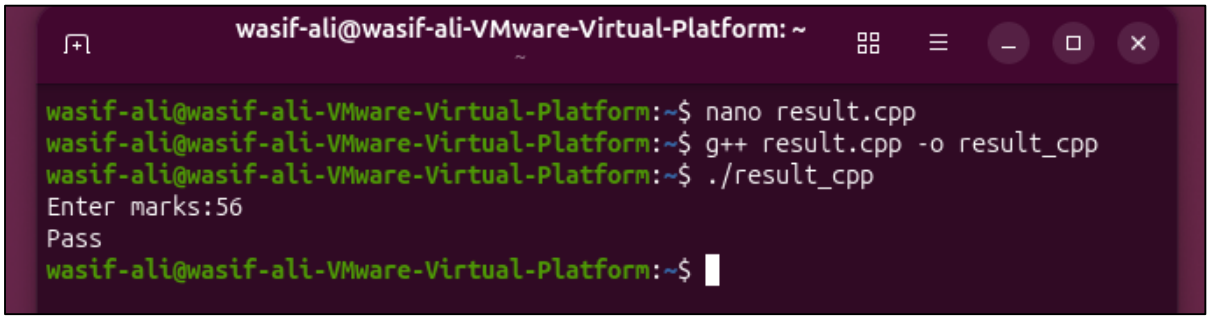


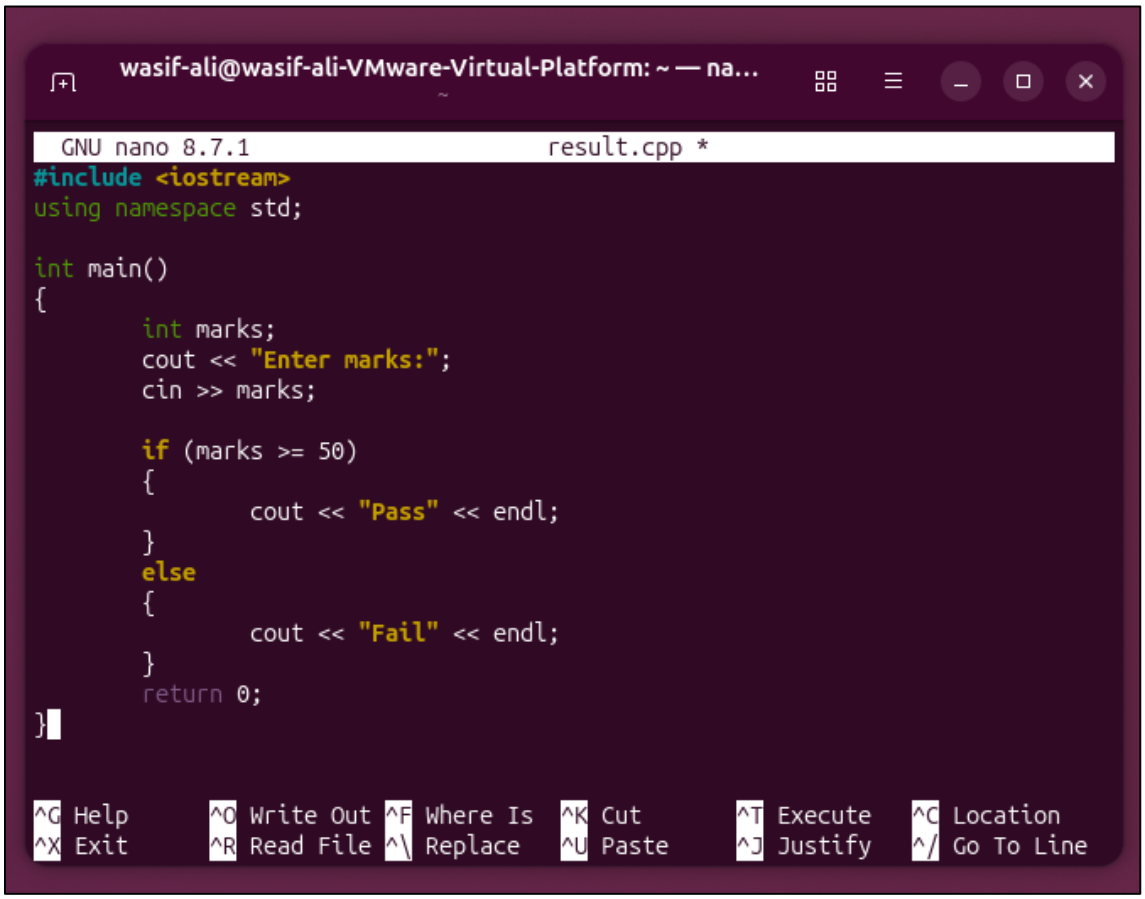




Commands & Code (C++ Program):







Lab Activity 5 — Debugging a C++ Program

The following C++ program contains a syntax error.

```
#include <iostream>

using namespace std;


int main()

{

   int a = 10;

   int b = 20


   cout << "Sum = " << a + b << endl;


   return 0;

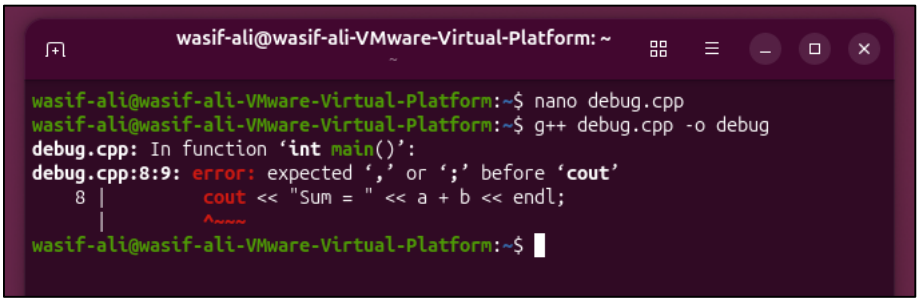
}
```

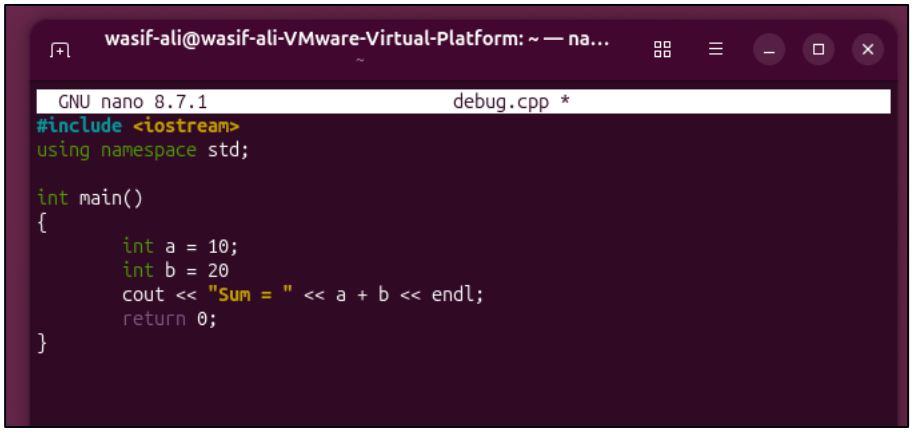
1. **Create** a file named debug.cpp.
2. **Insert** the given program.
3. **Compile** the program using g++.
4. **Identify** the compilation error.
5. **Modify** the program to correct the error.
6. **Compile** the program again.
7. **Execute** the program.
8. **Verify** the output.

Working:

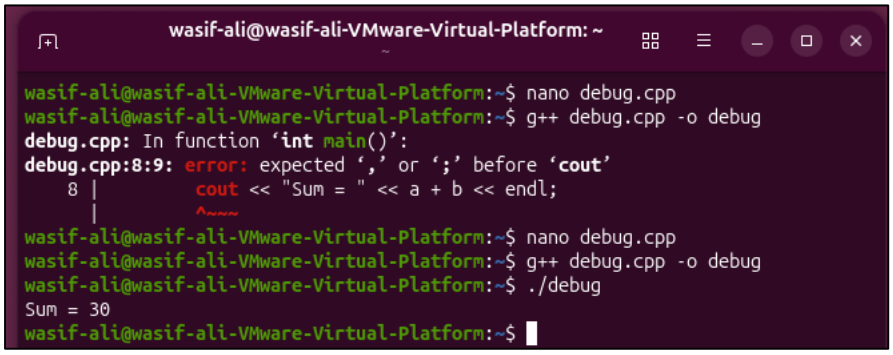
In this activity, we identified and corrected a syntax compilation error in a C++ source file named debug.cpp. The original code failed to compile due to a missing semicolon ; after the variable declaration int b = 20. We edited the file in nano, added the required semicolon, recompiled using g++, and executed the fixed program to verify the correct sum output.

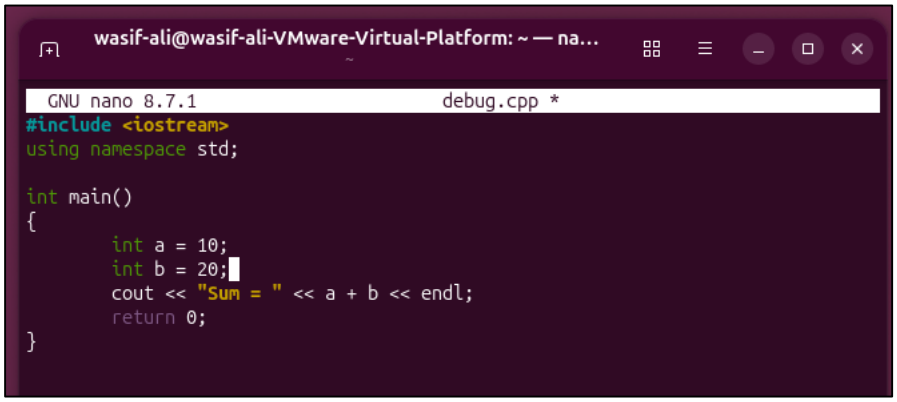
Given Code:





Corrected code:





Lab Activity 6 — Compile Using CC

Create a simple C program and compile it using the cc command.

1. **Create** a file named ccprogram.c.
2. **Insert** a C program that displays a message.
3. **Save** the file.
4. **Compile** the program using the cc compiler command.
5. **Execute** the generated executable.
6. **Verify** the output.

Working:

In this activity, we created a simple C program named ccprogram.c. Instead of using the gcc command directly, we compiled the file using cc, which is the system's default C compiler interface. Running the compiled binary demonstrated that the cc command successfully processes and builds C programs in Ubuntu.

